CardioRisk MLOps

End-to-End ML Model Development, CI/CD, Containerization and Production Deployment

Educational implementation using the UCI Heart Disease Cleveland dataset

Student Name: _____
Student ID: _____
Course / Semester: _____
Repository URL: _____
Submission Date: _____

Local API access: http://localhost:8000
API documentation: http://localhost:8000/docs

Disclaimer: This model is an academic demonstration and is not intended for medical diagnosis or treatment.

1. Problem Statement and Objectives

The objective is to engineer a reproducible binary classification service that estimates whether the UCI target indicates presence or absence of heart disease. The project treats the model as one component of a complete software system, including source-controlled preprocessing, automated tests, experiment tracking, CI/CD, containerized serving, Kubernetes deployment, and operational monitoring.

- Automate acquisition and validation of the processed Cleveland dataset.
- Compare Logistic Regression and Random Forest using stratified cross-validation.
- Track parameters, metrics and artifacts in MLflow.
- Package the identical preprocessing-model pipeline used during training.
- Expose validated JSON inference through FastAPI.
- Demonstrate repeatable Docker and Minikube deployment with monitoring.

Success criteria

All scripts execute from a clean Python 3.11 environment; tests and linting block failed CI runs; the API returns prediction, confidence, version and request ID; and deployment health and latency are observable.

2. Dataset, Acquisition and Governance

The UCI Heart Disease database contains four source databases. This project uses the widely adopted processed Cleveland subset with 303 records, 13 predictors and the original num outcome. The download script converts question-mark tokens to missing values and converts num into target, where 0 means absence and values 1-4 mean presence.

- Source: UCI Machine Learning Repository, Heart Disease dataset.
- DOI: 10.24432/C52P4X.
- License: Creative Commons Attribution 4.0.
- Raw data is retained in data/raw; cleaned data is written to data/processed.
- A SHA-256 checksum is printed to support dataset traceability.
- No names or direct identifiers are used by the model.

Data quality controls

The loader verifies the complete schema and binary target. Numeric conversion is explicit. Missing values are not filled in the CSV because imputation is learned inside each training fold, preventing information from the validation partitions leaking into preprocessing.

3. Exploratory Data Analysis

The EDA script generates class-balance, distribution and correlation views. The notebook provides an interactive companion while the script supports repeatable automation in CI. The analysis should describe distributions and data quality without implying clinical causation.

- Class count and percentage for target 0 and 1.
- Histograms for age, resting blood pressure, cholesterol, maximum heart rate and oldpeak.
- Correlation heatmap for numeric representations.
- Missing-value count, especially ca and thal.
- Outlier review using domain-aware inspection rather than automatic deletion.

Interpretation principles

Categorical fields are stored numerically but are not assumed to have continuous distance. Correlation is descriptive rather than causal. Because the sample is small, charts and metrics must be interpreted with uncertainty and tested through stratified resampling.

4. Feature Engineering and Model Development

A ColumnTransformer separates true numeric features from coded categorical features. Numeric features receive median imputation and standardization. Categorical features receive most-frequent imputation and one-hot encoding with unknown-category tolerance. The transformer is embedded in the fitted model Pipeline.

Candidate models

- Logistic Regression: interpretable linear baseline with balanced class weighting.
- Random Forest: nonlinear ensemble with 300 trees and balanced class weighting.

Evaluation design

The data is split 80/20 using target stratification and random state 42. Model comparison uses five-fold StratifiedKFold cross-validation. ROC-AUC is the primary selection metric, with recall, precision, F1 and accuracy retained for balanced interpretation. The holdout set is used after model selection.

The training script selects the highest mean cross-validated ROC-AUC and writes the complete pipeline to models/heart_disease_pipeline.joblib. Actual metrics must be captured from the student execution and should not be invented in the report.

5. Experiment Tracking and Reproducibility

MLflow records one run per candidate model. Each run logs model identity, fold count, cross-validated ROC-AUC mean and standard deviation, and holdout accuracy, precision, recall, F1 and ROC-AUC. The fitted scikit-learn pipeline is stored as an MLflow artifact with an input example.

- Random state and split strategy are fixed in source control.
- Dependencies are pinned in requirements.txt and mirrored by environment.yml.
- The exact serialized pipeline includes preprocessing and classification.
- Model metadata records selected model, semantic version and metrics.
- Artifacts can be inspected through mlflow ui --backend-store-uri ./mlruns.

Reproducibility limitation

Package pinning and deterministic seeds improve repeatability, but tree execution, operating system libraries and future package availability can still affect bit-level equivalence. Docker provides the strongest runtime isolation in this submission.

6. Automated Testing and CI/CD

Pytest covers schema validation, rejection of non-binary targets, missing-value preprocessing, probability bounds, API health, and invalid payload validation. Ruff checks source quality. GitHub Actions executes on pushes, pull requests and manual dispatch.

- Install pinned Python 3.11 dependencies.
- Run Ruff and fail on lint violations.
- Run Pytest with coverage output.
- Download and clean the official dataset.
- Train and persist the selected model.
- Upload model, figures and coverage results as workflow artifacts.
- Build the Docker image and execute a container health check.

Failure behavior

GitHub Actions stops the workflow when any command exits non-zero. A failed lint check, unit test, data download, training process, image build or health call therefore prevents a falsely successful pipeline.

7. API Packaging and Containerization

FastAPI exposes /health, /predict and /metrics. Pydantic validates plausible structural ranges and rejects incomplete payloads with HTTP 422. The prediction response contains a binary result, cautious model-estimated risk label, probability confidence, model version and request ID.

- The container uses python:3.11-slim.
- Requirements are installed before source copy to improve caching.
- The process runs as a non-root app user.
- A Docker HEALTHCHECK calls /health.
- The model is loaded once at API startup.
- Raw clinical values are not written to logs.

Local verification

Build with docker build -t cardiorisk-api:1.0.0 . and run with docker run --rm -p 8000:8000 cardiorisk-api:1.0.0. Send sample_request.json to /predict. The provided repository contains commands in README.md.

8. Kubernetes Deployment Architecture



The Kubernetes manifest creates a dedicated namespace, two FastAPI replicas, resource requests and limits, rolling updates, and liveness/readiness probes. A ClusterIP service exposes the pods within the cluster; port-forwarding supplies deterministic local access. An optional Ingress manifest maps cardiorisk.local when an Ingress controller is enabled.

- Image: cardiorisk-api:1.0.0 loaded into Minikube.
- Replicas: 2 for availability demonstration.
- Readiness: /health every 10 seconds after 5-second delay.
- Liveness: /health every 20 seconds after 15-second delay.
- Local endpoint: http://localhost:8000 after kubectl port-forward.

9. Monitoring, Logging and Operational Safety

The API exposes Prometheus-format metrics and structured JSON operational logs. The Prometheus configuration scrapes the Kubernetes service every 15 seconds. Grafana can visualize traffic, latency, failures and model availability.

- cardiorisk_requests_total: request volume.
- cardiorisk_prediction_errors_total: serving failures.
- cardiorisk_request_latency_seconds: latency histogram for percentile panels.
- Health response: model-loaded status and version.
- Logs: request ID, endpoint and latency only; no health feature payload.

Recommended dashboard and alerts

Display requests per minute, error rate, P50/P95 latency, pod CPU, pod memory and readiness. For demonstration, alert when error rate exceeds 5% for five minutes or P95 latency exceeds one second. These are engineering thresholds, not medical performance guarantees.

10. Results, Limitations and Submission Evidence

This repository provides a complete, executable framework, but authentic run outputs remain environment-dependent. The student should execute the pipeline, record the resulting metrics, and insert genuine screenshots into the prepared folders before final submission. No deployment or experimental evidence is fabricated in this package.

Key limitations

- Small historical dataset with limited population diversity.
- Potential missingness and coding ambiguity in ca and thal.
- Cross-validation reduces but does not remove sampling uncertainty.
- Probability confidence is not the same as clinical certainty.
- No external validation, calibration study, fairness review or regulatory assessment.

Evidence checklist

- MLflow comparison and selected run.
- Successful GitHub Actions workflow.
- Docker build, health and prediction calls.
- Kubernetes pods, service and endpoint test.
- Prometheus target and Grafana dashboard.
- Repository link and short demonstration video link.

Conclusion

CardioRisk demonstrates how reproducible preprocessing, controlled experimentation, automated quality gates, portable packaging, deployment probes and observability combine into an MLOps lifecycle. The design deliberately keeps one fitted pipeline from training through serving to minimize training-serving skew.

References

UCI Machine Learning Repository. Heart Disease. DOI 10.24432/C52P4X. CC BY 4.0.
scikit-learn, MLflow, FastAPI, Docker, Kubernetes, Prometheus and Grafana official documentation.